



Sesión 3: Introducción a FastAPI y APIs REST

Construiremos una API para gestionar solicitudes de soporte técnico. Aprenderemos desde los fundamentos de REST hasta la validación, documentación y buenas prácticas.

❓ ¿Qué diferencia existe entre una función de Python y un endpoint de una API?

¿Qué es una API REST?

Una API permite que diferentes sistemas intercambien información mediante solicitudes y respuestas estructuradas.



Principios REST básicos

- Los recursos se representan mediante sustantivos
- Se utilizan métodos HTTP para las acciones
- Las respuestas incluyen códigos de estado
- Los datos se intercambian en formato JSON
- Cada solicitud contiene toda la información necesaria

□ **Ejemplo:** GET /api/v1/tickets/1001 → JSON con los datos de la solicitud.

Nuestro primer proyecto con FastAPI

Estructura del proyecto

```
helpdesk_api/  
├── .venv/  
├── app/  
│   ├── __init__.py  
│   └── main.py  
└── requirements.txt
```

Instalación y ejecución

```
pip install fastapi "uvicorn[standard]"  
python -m uvicorn app.main:app --reload
```

Accede a: `http://127.0.0.1:8000/docs`

Primer endpoint

```
from fastapi import FastAPI  
  
app = FastAPI(  
    title="HelpDesk API",  
    version="1.0.0",  
)  
  
@app.get("/")  
def home():  
    return {  
        "message": "HelpDesk API funcionando"  
    }
```

FastAPI genera automáticamente la documentación OpenAPI en `/docs` y `/redoc`.

Endpoints, rutas y métodos HTTP

Un endpoint combina **método HTTP + ruta + función**. Cada combinación define una operación única sobre un recurso.

Acción	Método	Ruta	Respuesta
Listar	GET	/tickets	200
Consultar	GET	/tickets/{id}	200
Registrar	POST	/tickets	201
Actualizar	PATCH	/tickets/{id}	200
Eliminar	DELETE	/tickets/{id}	204

 **PUT vs PATCH:** PUT reemplaza el recurso completo; PATCH modifica solo los campos indicados. Para cambiar el estado, usaremos PATCH.

Parámetros de ruta y Query Strings

Los parámetros de ruta **identifican** un recurso concreto; los query strings **filtran** o modifican la consulta.

Path Parameter

```
@app.get("/tickets/{ticket_id}")
def get_ticket(ticket_id: int):
    return {"ticket_id": ticket_id}
```

GET /tickets/1001

- Identifica un recurso concreto
- Generalmente obligatorio
- Forma parte de la ruta

Query String

```
@app.get("/tickets")
def list_tickets(
    priority: str | None = None,
    status: str | None = None,
):
    ...
```

GET /tickets?priority=Alta&status=Pendiente

- Filtra, ordena o pagina resultados
- Generalmente opcional
- Se coloca después de ?

Validación con Pydantic

Pydantic define la estructura y las reglas de los datos de entrada, garantizando contratos fiables desde el primer momento.

```
from pydantic import BaseModel, Field
```

```
class TicketCreate(BaseModel):  
    title: str = Field(min_length=5, max_length=120)  
    description: str = Field(min_length=10, max_length=500)  
    priority: str = Field(default="Media")  
    requester_email: str
```

Verificación de tipos

Garantiza que cada campo tenga el tipo correcto

Restricciones

Longitudes mínimas y máximas configurables

Valores por defecto

Campos opcionales con valores predeterminados

Documentación automática

Los esquemas aparecen en Swagger sin esfuerzo

Controlar lo que recibimos y devolvemos

Esquemas de entrada y salida

```
class TicketCreate(BaseModel):
    title: str
    description: str
    priority: str
    requester_email: str

class TicketResponse(BaseModel):
    id: int
    title: str
    priority: str
    status: str
```

Endpoint con response_model

```
@app.post(
    "/tickets",
    response_model=TicketResponse,
    status_code=201,
)
def create_ticket(data: TicketCreate):
    ticket = {
        "id": len(tickets) + 1,
        "title": data.title,
        "priority": data.priority,
        "status": "Pendiente",
    }
    tickets.append(ticket)
    return ticket
```



`response_model` filtra campos internos, define el contrato de salida y mejora la documentación automática.

Manejo de errores y códigos HTTP

Error 404 en FastAPI

```
from fastapi import HTTPException

@app.get("/tickets/{ticket_id}")
def get_ticket(ticket_id: int):
    for ticket in tickets:
        if ticket["id"] == ticket_id:
            return ticket
    raise HTTPException(
        status_code=404,
        detail="Solicitud no encontrada"
    )
```

Códigos de estado frecuentes

200	Operación exitosa
201	Recurso creado
204	Éxito sin contenido
400	Solicitud incorrecta
404	Recurso no encontrado
409	Conflicto con el estado
422	Error de validación
500	Error interno del servidor



Nunca devuelvas `status: 200` con un mensaje de error. Usa siempre el código HTTP adecuado.

Documentación, trazabilidad y evolución de la API

Swagger y OpenAPI

FastAPI genera automáticamente documentación interactiva con rutas, parámetros, esquemas y herramienta de pruebas en `/docs`.

Middleware de trazabilidad

```
from time import perf_counter

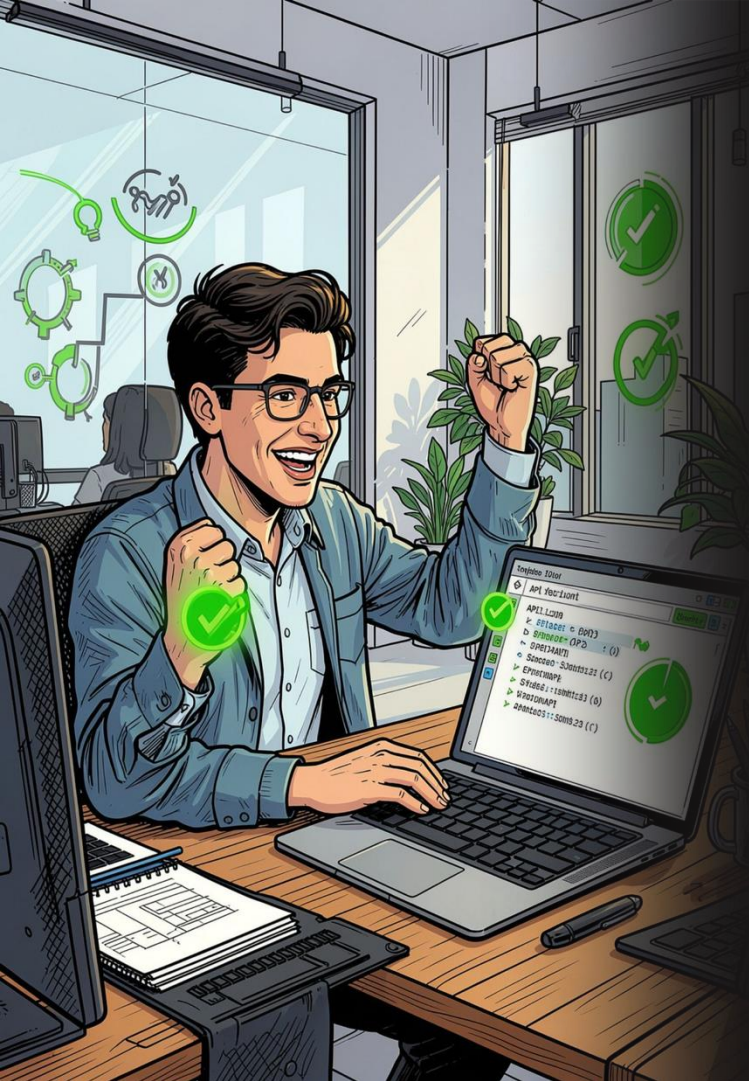
@app.middleware("http")
async def add_process_time(request, call_next):
    start = perf_counter()
    response = await call_next(request)
    elapsed = perf_counter() - start
    response.headers["X-Process-Time"] =
    str(elapsed)
    return response
```

Versionamiento y buenas prácticas

```
app.include_router(
    tickets_router,
    prefix="/api/v1"
)
```

Rutas: `/api/v1/tickets` → `/api/v2/tickets`

- Sustantivos en plural en las rutas
- Métodos HTTP correctos para cada acción
- Códigos de estado adecuados
- Validar entradas con Pydantic
- No exponer detalles internos
- Versionar la API desde el inicio



Reto integrador: HelpDesk API v1

Implementa los siguientes endpoints con validación Pydantic, códigos HTTP correctos y middleware de trazabilidad.

1

POST /api/v1/tickets

Crear solicitud con validación de campos

2

GET /api/v1/tickets

Listar con filtros opcionales por prioridad y estado

3

GET /api/v1/tickets/{id}

Consultar por identificador; devolver 404 si no existe

4

PATCH /api/v1/tickets/{id}/status

Actualizar estado; rechazar valores no reconocidos



¿Qué componente sustituiremos en la siguiente sesión para almacenar permanentemente las solicitudes?